

Linear-Time Sorting Algorithms:

Counting Sort, Radix Sort & Bucket Sort

Sorting In Linear Time

- Counting sort
 - No comparisons between elements!
 - **But**...depends on assumption about the numbers being sorted
 - We assume numbers are in the range $1, \dots, k$
 - The algorithm:
 - Input: $A[1..n]$, where $A[j] \in \{1, 2, 3, \dots, k\}$
 - Output: $B[1..n]$, sorted (notice: not sorting in place)
 - Also: Array $C[1..k]$ for auxiliary storage

Counting Sort

COUNTING-SORT(A, B, k)

```
1  for  $i \leftarrow 0$  to  $k$ 
2      do  $C[i] \leftarrow 0$ 
3  for  $j \leftarrow 1$  to  $\text{length}[A]$ 
4      do  $C[A[j]] \leftarrow C[A[j]] + 1$ 
5   $\triangleright C[i]$  now contains the number of elements equal to  $i$ .
6  for  $i \leftarrow 1$  to  $k$ 
7      do  $C[i] \leftarrow C[i] + C[i - 1]$ 
8   $\triangleright C[i]$  now contains the number of elements less than or equal to  $i$ .
9  for  $j \leftarrow \text{length}[A]$  downto 1
10     do  $B[C[A[j]]] \leftarrow A[j]$ 
11      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

COUNTING-SORT assumes that each of the input elements is an integer in the range 0 to k , inclusive.

$A[1..n]$ is the input array, $B[1..n]$ is the output array, $C[0..k]$ is a temporary working array.

Counting Sort

COUNTING-SORT(A, B, k)

```
1  for  $i \leftarrow 0$  to  $k$ 
2      do  $C[i] \leftarrow 0$ 
3  for  $j \leftarrow 1$  to  $\text{length}[A]$ 
4      do  $C[A[j]] \leftarrow C[A[j]] + 1$ 
5  ▷  $C[i]$  now contains the number of elements equal to  $i$ .
6  for  $i \leftarrow 1$  to  $k$ 
7      do  $C[i] \leftarrow C[i] + C[i - 1]$ 
8  ▷  $C[i]$  now contains the number of elements less than or equal to  $i$ .
9  for  $j \leftarrow \text{length}[A]$  downto 1
10     do  $B[C[A[j]]] \leftarrow A[j]$ 
11         $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

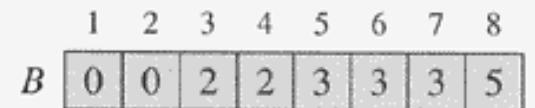
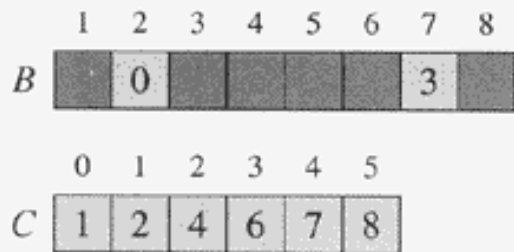
Takes time $O(k)$

Takes time $O(n)$

What will be the running time?

- Total time: $O(n + k)$
 - Usually, $k = O(n)$
 - Thus counting sort runs in $O(n)$ time

Counting Sort



Counting Sort

- Cool! *Why don't we always use counting sort?*
- Because it depends on range k of elements
- *Could we use counting sort to sort 32 bit integers? Why or why not?*
- Answer: no, k too large ($2^{32} = 4,294,967,296$)

Radix Sort

- Radix sort:
 - Assumption: input has d digits ranging from 0 to k
 - Basic idea:
 - Sort elements by digit starting with *least* significant
 - Use a stable sort (like counting sort) for each stage

```
RADIX-SORT( $A, d$ )
```

```
1  for  $i \leftarrow 1$  to  $d$ 
```

```
2      do use a stable sort to sort array  $A$  on digit  $i$ 
```

A **stable sort** is one where items with the same value appear in the same order in the output as they appeared in the input.

Is Counting-Sort stable?!!

Radix Sort

RADIX-SORT(A, d)

```
1  for  $i \leftarrow 1$  to  $d$ 
2      do use a stable sort to sort array  $A$  on digit  $i$ 
```

329	720	720	329
457	355	329	355
657	436	436	436
839	457	839	457
436	657	355	657
720	329	457	720
355	839	657	839

Radix Sort

RADIX-SORT(A, d)

1 **for** $i \leftarrow 1$ **to** d

2 **do** use a stable sort to sort array A on digit i

Lemma 8.3: Given n d -digit numbers in which each digit can take up to k possible values. Radix-Sort correctly sorts these numbers in $\Theta(d(n+k))$ time.

Proof: Perform Counting-Sort on each column.

$$\begin{aligned}\text{Running time} &= \Theta(n+k) * \text{No. of columns} \\ &= \Theta(d(n+k)) \quad \blacklozenge\end{aligned}$$

When d is constant and $k = O(n)$,

$$\text{Running time} = O(n)$$

Bucket Sort

- Bucket sort
 - Assumption: input is n reals from $[0, 1)$
 - Basic idea:
 - Create n linked lists (*buckets*) to divide interval $[0,1)$ into subintervals of size $1/n$
 - Add each input element to appropriate bucket and sort buckets with insertion sort
 - Uniform input distribution $\rightarrow O(1)$ bucket size
 - Therefore the expected total time is $O(n)$

Bucket Sort

BUCKET-SORT(A)

```
1   $n \leftarrow \text{length}[A]$ 
2  for  $i \leftarrow 1$  to  $n$ 
3      do insert  $A[i]$  into list  $B[\lfloor nA[i] \rfloor]$ 
4  for  $i \leftarrow 0$  to  $n - 1$ 
5      do sort list  $B[i]$  with insertion sort
6  concatenate the lists  $B[0], B[1], \dots, B[n - 1]$  together in order
```

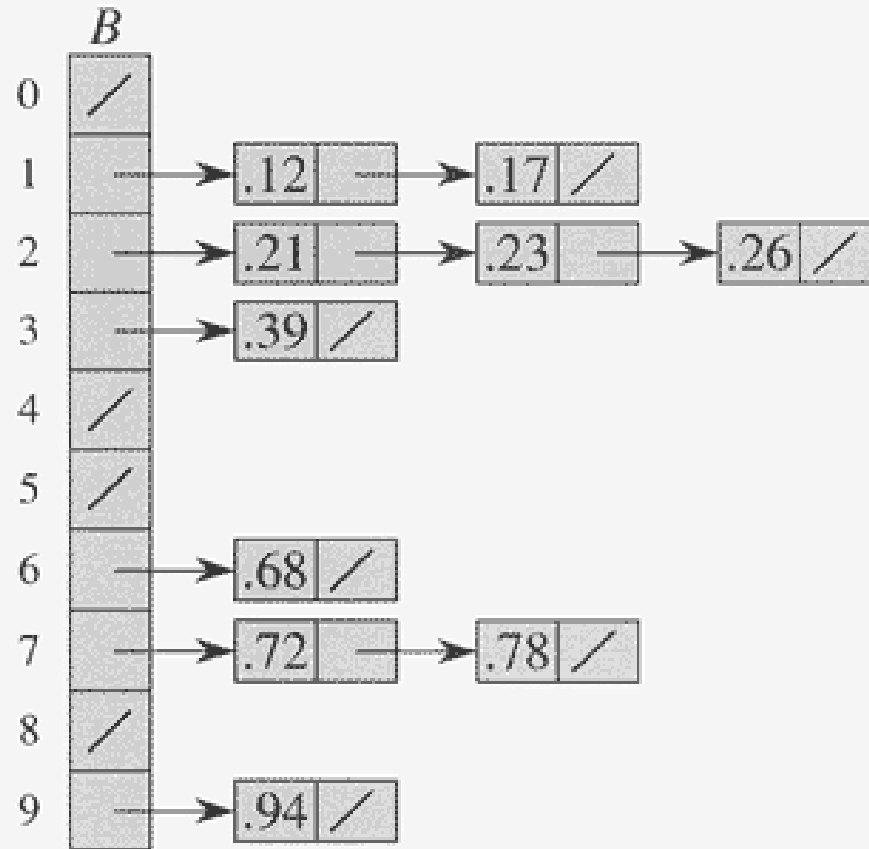
BUCKET-SORT assumes each input element $A[i]$ lies in $0 \leq A[i] < 1$ (i.e., normalized).

$A[1..n]$ is the input array, $B[0..n-1]$ is an auxiliary array of linked lists (=buckets).

Bucket Sort

	<i>A</i>
1	.78
2	.17
3	.39
4	.26
5	.72
6	.94
7	.21
8	.12
9	.23
10	.68

(a)



(b)

Bucket Sort

The running time is

$$T(n) = \Theta(n) + \sum_{i=0..n-1} O(n_i^2),$$

where n_i is the random variable representing the number of elements in bucket $B[i]$. [note that insertion sort is quadratic-time]

Therefore,

$$\begin{aligned} E[T(n)] &= E[\Theta(n) + \sum_{i=0..n-1} O(n_i^2)] = \Theta(n) + \sum_{i=0..n-1} E[O(n_i^2)] \\ &= \Theta(n) + \sum_{i=0..n-1} O(E[n_i^2]) \end{aligned}$$

Claim: $E[n_i^2] = 2 - 1/n$

If the claim is proved:

$$\begin{aligned} E[T(n)] &= \Theta(n) + \sum_{i=0..n-1} O(2 - 1/n) \\ &= \Theta(n) + n * O(2 - 1/n) \\ &= \Theta(n) \end{aligned}$$